

MongoDB Terminal Practice Session

All CRUD Commands + Query Operators + Library Use Case

1. Database & Collection Setup

```
# Start mongosh (MongoDB Shell)
mongosh
Current Mongosh Log ID: 64f3a1b2c3d4e5f6a7b8c9d0
Connecting to: mongodb://127.0.0.1:27017/
Using MongoDB: 7.0.2

# Switch to (or create) studentDB
use studentDB
switched to db studentDB

# Explicitly create a collection
db.createCollection("students")
{ ok: 1 }

# List all databases
show dbs
admin 40.00 KiB
config 60.00 KiB
local 72.00 KiB
studentDB 8.00 KiB

# List collections in current database
show collections
students
```

2. Insert Operations

```
# insertOne() – add a single document
db.students.insertOne({
  name: "Alice Johnson",
  age: 22,
  course: "MERN Stack",
  status: "active",
  enrolledDate: new Date("2024-01-15")
})
{
  acknowledged: true,
  insertedId: ObjectId("64f3a1b2c3d4e5f6a7b8c9d1")
}
```

```
# insertMany() – add multiple documents
db.students.insertMany([
  { name: "Bob Smith", age: 25, course: "Python & Django", status: "active" },
  { name: "Carol Davis", age: 20, course: "MERN Stack", status: "completed" },
  { name: "David Lee", age: 28, course: "Data Science", status: "inactive" },
  { name: "Emma Wilson", age: 23, course: "MERN Stack", status: "active" }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId("64f3a1b2c3d4e5f6a7b8c9d2"),
    '1': ObjectId("64f3a1b2c3d4e5f6a7b8c9d3"),
    '2': ObjectId("64f3a1b2c3d4e5f6a7b8c9d4"),
    '3': ObjectId("64f3a1b2c3d4e5f6a7b8c9d5")
  }
}
```

3. Read Operations

```
# Fetch all documents
db.students.find()
[
  { _id: ObjectId("...d1"), name: "Alice Johnson", age: 22, course: "MERN Stack", status: "active" },
  { _id: ObjectId("...d2"), name: "Bob Smith", age: 25, course: "Python & Django", status: "active" },
  { _id: ObjectId("...d3"), name: "Carol Davis", age: 20, course: "MERN Stack", status: "completed" },
  { _id: ObjectId("...d4"), name: "David Lee", age: 28, course: "Data Science", status: "inactive" },
  { _id: ObjectId("...d5"), name: "Emma Wilson", age: 23, course: "MERN Stack", status: "active" }
]

# Filter – students enrolled in MERN Stack
db.students.find({ course: "MERN Stack" })
[
  { name: "Alice Johnson", age: 22, course: "MERN Stack", status: "active" },
  { name: "Carol Davis", age: 20, course: "MERN Stack", status: "completed" },
  { name: "Emma Wilson", age: 23, course: "MERN Stack", status: "active" }
]

# Projection – show only name and course, hide _id
db.students.find({ course: "MERN Stack" }, { name: 1, course: 1, _id: 0 })
[
  { name: "Alice Johnson", course: "MERN Stack" },
  { name: "Carol Davis", course: "MERN Stack" },
  { name: "Emma Wilson", course: "MERN Stack" }
]
```

```

]
# findOne() – returns first matching document
db.students.findOne({ name: "Alice Johnson" })
{ _id: ObjectId("...d1"), name: "Alice Johnson", age: 22, course: "MERN Stack", status:
"active" }
# Sort by age descending, limit 3 results
db.students.find().sort({ age: -1 }).limit(3)
[
{ name: "David Lee", age: 28, course: "Data Science" },
{ name: "Bob Smith", age: 25, course: "Python & Django" },
{ name: "Emma Wilson", age: 23, course: "MERN Stack" }
]
# Count all documents
db.students.countDocuments()
5

```

4. Update Operations

```

# updateOne() – change Alice's status to 'completed'
db.students.updateOne(
{ name: "Alice Johnson" },
{ $set: { status: "completed" } }
)
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }
# updateMany() – set all MERN Stack students to 'active'
db.students.updateMany(
{ course: "MERN Stack" },
{ $set: { status: "active" } }
)
{ acknowledged: true, matchedCount: 3, modifiedCount: 3 }
# $inc – increment Bob's age by 1
db.students.updateOne(
{ name: "Bob Smith" },
{ $inc: { age: 1 } }
)
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }
# $push – add a tag to Carol's document
db.students.updateOne(
{ name: "Carol Davis" },
{ $push: { tags: "scholarship" } }
)
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }

```

5. Delete Operations

```

# deleteOne() — remove a specific student
db.students.deleteOne({ name: "David Lee" })
{ acknowledged: true, deletedCount: 1 }

# deleteMany() — remove all inactive students
db.students.deleteMany({ status: "inactive" })
{ acknowledged: true, deletedCount: 0 }

# Delete ALL documents (practice only!)
db.students.deleteMany({})
{ acknowledged: true, deletedCount: 4 }

# Drop entire collection
db.students.drop()
true

```

6. Query Operators — \$gt, \$lt, \$in, \$and, \$or, \$exists

```

# Re-insert sample data
db.students.insertMany([
  { name: "Alice", age: 22, course: "MERN Stack", status: "active" },
  { name: "Bob", age: 26, course: "Python & Django", status: "active" },
  { name: "Carol", age: 20, course: "MERN Stack", status: "completed" },
  { name: "David", age: 28, course: "Data Science", status: "inactive" },
  { name: "Emma", age: 23, course: "MERN Stack", status: "active" }
])
{ acknowledged: true, insertedIds: { '0': ..., '1': ..., '2': ..., '3': ..., '4': ... } }

# $gt — students older than 22
db.students.find({ age: { $gt: 22 } })
[ { name: "Bob", age: 26, ... }, { name: "David", age: 28, ... }, { name: "Emma", age: 23, ... } ]

# $lt — students younger than 23
db.students.find({ age: { $lt: 23 } })
[ { name: "Alice", age: 22, ... }, { name: "Carol", age: 20, ... } ]

# $gt + $lt — age between 21 and 26
db.students.find({ age: { $gt: 21, $lt: 26 } })
[ { name: "Alice", age: 22, ... }, { name: "Emma", age: 23, ... } ]

# $in — students in MERN Stack or Data Science
db.students.find({ course: { $in: ["MERN Stack", "Data Science"] } })
[ { name: "Alice", ... }, { name: "Carol", ... }, { name: "David", ... }, { name: "Emma", ... } ]

# $and — active MERN Stack students older than 21
db.students.find({
  $and: [
    { course: "MERN Stack" },
    { status: "active" },
    { age: { $gt: 21 } }
  ]
})

```

```

]
})
[ { name: "Alice", age: 22, course: "MERN Stack", status: "active" },
  { name: "Emma", age: 23, course: "MERN Stack", status: "active" } ]
# $or – completed status OR age > 25
db.students.find({
  $or: [
    { status: "completed" },
    { age: { $gt: 25 } }
  ]
})
[ { name: "Bob", age: 26, ... }, { name: "Carol", status: "completed", ... }, { name: "David",
age: 28, ... } ]
# $exists – students who have an enrolledDate field
db.students.find({ enrolledDate: { $exists: true } })
[] // (no enrolledDate was set in this insert – returns empty)
# $exists: false – students WITHOUT a tags field
db.students.find({ tags: { $exists: false } })
[ { name: "Alice", ... }, { name: "Bob", ... }, { name: "Carol", ... }, ... ]

```

7. Use Case — Library Management System

```

# Setup
use libraryDB
switched to db libraryDB
# Insert books
db.books.insertMany([
  { title: "The Pragmatic Programmer", author: "Andrew Hunt", genre: "Technology", year: 1999,
copies: 5, available: 3 },
  { title: "Clean Code", author: "Robert C. Martin", genre: "Technology", year: 2008, copies: 8,
available: 0 },
  { title: "Sapiens", author: "Yuval Harari", genre: "History", year: 2011, copies: 4,
available: 4 },
  { title: "Atomic Habits", author: "James Clear", genre: "Self-Help", year: 2018, copies:
10,available: 7 }
])
{ acknowledged: true, insertedIds: { '0': ..., '1': ..., '2': ..., '3': ... } }
# Search: all available books (at least 1 copy)
db.books.find({ available: { $gt: 0 } }, { title: 1, available: 1, _id: 0 })
[
  { title: "The Pragmatic Programmer", available: 3 },
  { title: "Sapiens", available: 4 },
  { title: "Atomic Habits", available: 7 }
]
# Search: Technology books after year 2000

```

```
db.books.find({ $and: [ { genre: "Technology" }, { year: { $gt: 2000 } } ] })
[ { title: "Clean Code", author: "Robert C. Martin", genre: "Technology", year: 2008, ... } ]

# Search: books fully checked out
db.books.find({ available: 0 }, { title: 1, _id: 0 })
[ { title: "Clean Code" } ]

# Update: member borrows The Pragmatic Programmer
db.books.updateOne(
  { title: "The Pragmatic Programmer" },
  { $inc: { available: -1 } }
)
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }

# Update: add 3 new copies of Clean Code
db.books.updateOne(
  { title: "Clean Code" },
  { $inc: { copies: 3, available: 3 } }
)
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }

# Update: mark all Self-Help books as featured
db.books.updateMany(
  { genre: "Self-Help" },
  { $push: { tags: "featured" } }
)
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }

# Delete: remove a decommissioned book
db.books.deleteOne({ title: "Sapiens" })
{ acknowledged: true, deletedCount: 1 }

# Verify final state
db.books.find({}, { title: 1, available: 1, _id: 0 })
[
  { title: "The Pragmatic Programmer", available: 2 },
  { title: "Clean Code", available: 3 },
  { title: "Atomic Habits", available: 7 }
]
```